Software Requirements Specification (SRS)

Huston-Tillotson University
Software Engineering I

Software Requirements Specification (SRS)

PROJECT TITLE: Smart Copy Book

PROJECT DEVELOPER: Nabil Ajiboye

PROJECT REPOSITORY : https://github.com/MNA313

Table of Contents

REVISION HISTORY

1- INTRODUCTION

1.1- Purpose
1.2- Document convention
1.3- Intended Audience and Reading Suggestions
1.4- Project Scope
1.5- References

2- OVERALL DESCRIPTION

2.1- Product Perspective
2.2- Product Features
2.3- User Classes and Characteristics
2.4- Operating Environment
2.5 - Design and Implementation Constraints
2.6 - User Documentation
2.7- Assumptions and Dependencies

3- SYSTEM FEATURES

3.1- Handwriting Recognition
3.2 - Voice-to-Text Transcription
3.3 - Text-to-Speech Playback
3.4 - AI-Powered Summarization
3.5 - Study Flashcards
3.6 - Tagging and Search
3.7 - Data Synchronization
3.8 - User Authentication
3.9 - Collaboration and Sharing
3.10- Multilingual Support

4- EXTERNAL INTERFACE REQUIREMENT

5- OTHER NONFUNCTIONAL REQUIREMENTS

6- OTHER REQUIREMENTS

Appendix A: Glossary
Appendix B: Analysis Models
Appendix C: Issues List

Revision History

Name	Date	Reason for Changes	Version
Nabil Ajiboye	Nov 2025	Initial SRS Draft for Smart Copy Book App	1.0

1. INTRODUCTION

1.1- Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements for the Smart Copy Book App. This document provides a detailed description of the system's objectives, user needs, and overall functionality.

The Smart Copy Book App is designed to revolutionize digital note-taking by integrating handwriting recognition, speech-to-text, text-to-speech, AI-assisted feedback, and organization tools. It allows users to write or speak notes, which are then digitized, categorized, and enhanced with intelligent study tools. This SRS serves as a reference for developers, testers, and project stakeholders throughout the development lifecycle.

1.2 - Document Conventions

- Text styles: Headings use bold and hierarchical numbering. Code snippets and technical keywords appear in monospace font.
- Priority labels: (H) High, (M) Medium, (L) Low priority.
- Follows IEEE SRS 830 standard.

1.3 - Intended Audience and Reading Suggestions

- Developers: Focus on Sections 2, 3, and 4 for system design and feature implementation.
- Testers: Focus on Section 3 for functional requirements and Section 5 for non-functional validation criteria.
- Instructors/Reviewers: Focus on Sections 1 and 2 for scope and justification.

1.4- Project Scope

The Smart Copy Book App aims to digitize handwritten and spoken notes into intelligent, searchable, and interactive formats. Users can write using Apple Pencil or their voice, and the system automatically converts, organizes, and enhances content through AI. The project's scope includes the creation of a mobile and desktop (iOS/macOS) application, integrated with a Vapor backend for synchronization and AI services.

Goals:
- Enable seamless note-taking through handwriting or speech.
- Provide accurate handwriting and voice recognition.
- Offer AI-powered summaries, translations, and flashcards.
- Facilitate cloud synchronization and collaboration.
- Maintain user privacy with secure local data storage.

Benefits:
- Increases productivity and learning efficiency.
- Offers accessibility through voice and text interfaces.
- Provides a smarter way to learn and review handwritten material.

1.5 - References

- Apple Developer Documentation: PencilKit, Speech Framework, Vision Framework.
- Vapor Framework Documentation.
- Hugging Face Transformers & TrOCR documentation.
- IEEE 830-1998 Standard for SRS.

2. OVERALL DESCRIPTION

2.1- Product Perspective

The Smart Copy Book App is a hybrid mobile and desktop application that combines note-taking, handwriting recognition, speech input, and AI-powered organization. It interacts with a backend server (Vapor) for storage, synchronization, and advanced processing tasks like handwriting recognition (HTR) and natural language summarization.

2.2- Product Features

- Handwriting input and recognition via PencilKit + Vision API.
- Voice-to-text transcription using Apple Speech Framework.
- Text-to-speech note playback.
- AI-powered summarization and study flashcards.
- Note organization, tagging, and search.
- Secure local and cloud data storage.
- User authentication and personalization.
- Collaboration and sharing capabilities.

2.3- User Classes and Characteristics

User Class	Description	Technical Expertise
Students	Take notes, study with summaries, and organize content.	Low
Educators	Review and share notes, generate learning content.	Medium
Professionals	Use for meetings and productivity.	Medium
Developers/Admins	Maintain backend and AI services.	High

2.4- Operating Environment

- Platform: iOS 17+, macOS 14+
- Backend: Vapor 4 (Swift)

- Database: SQLite (local), PostgreSQL (server)
- AI Models: TrOCR, Whisper, Hugging Face Transformers

2.5- Design and Implementation Constraints

- Must comply with Apple App Store guidelines.
- Limited by device memory and processing power.
- Offline handwriting recognition may have reduced accuracy.
- Data privacy laws (GDPR, CCPA) must be followed.

2.6 - User Documentation

- User Manual (PDF)
- In-app onboarding and tutorials.
- Developer API documentation (Swagger for Vapor backend).

2.7- Assumptions and Dependencies

- Users possess devices with Pencil or microphone input.
- Server requires consistent uptime for AI services.
- Internet connectivity needed for synchronization and AI processing.

3. SYSTEM FEATURES

3.1 Handwriting Recognition

Description: Converts handwritten notes into editable digital text.
Priority: High
Functional Requirements:
- The app shall capture handwriting via PencilKit.
- The system shall convert handwriting to text using Vision or TrOCR API.
- The recognized text shall be editable and searchable.

3.2 Voice-to-Text Transcription

Description: Converts spoken input into written text.
Priority: High
Functional Requirements:
- Use Apple Speech Framework for real-time transcription.
- Store transcribed text as a new note entry.

3.3 Text-to-Speech Playback

Description: Reads notes aloud for accessibility and review.
Priority: Medium

Functional Requirements:
- Use AVSpeechSynthesizer for text playback.
- Allow users to select playback voice and speed.

3.4 AI-Powered Summarization

Description: Generates concise summaries and key points from notes.
Priority: High
Functional Requirements:
- Integrate Hugging Face summarization models.
- Provide one-click summary generation.

3.5 Study Flashcards

Description: Automatically generate flashcards from important note sections.
Priority: Medium
Functional Requirements:
- Identify key terms or questions from notes.
- Allow users to edit or add custom flashcards.

3.6 Tagging and Search

Description: Enables organization and retrieval of notes.
Priority: High
Functional Requirements:
- Users can tag notes manually or via AI keyword extraction.
- Provide fuzzy search with ranking.

3.7 Data Synchronization

Description: Sync notes between local and cloud storage.
Priority: High
Functional Requirements:
- Use REST API via Vapor backend.
- Implement conflict resolution and offline caching.

3.8 User Authentication

Description: Secures access via password or voice.
Priority: High
Functional Requirements:
- Standard email/password authentication.
- Optional voice authentication API.

3.9 Collaboration and Sharing

Description: Allows users to share and co-edit notes.
Priority: Medium

Functional Requirements:
- Support sharing via unique links or cloud sync.
- Track edits and contributors.

3.10 Multilingual Support

Description: Supports translation and input in multiple languages.
Priority: Low
Functional Requirements:
- Integrate translation API (Hugging Face / Google Translate).
- Detect input language automatically.

4. EXTERNAL INTERFACE REQUIREMENTS

- User Interfaces: SwiftUI-based, supports handwriting, microphone input, and dark mode.
- Hardware Interfaces: Apple Pencil and microphone.
- Software Interfaces: PencilKit, Speech, Vision, Vapor REST API.
- Communication Interfaces: HTTPS and WebSocket.

5. OTHER NONFUNCIONAL REQUIREMENTS

- Performance: Recognition <3s, transcription <2s, supports 10,000 notes.
- Safety: Auto-save and recovery features.
- Security: End-to-end encryption and token-based authentication.
- Quality Attributes: Usability, reliability, scalability, maintainability.

6. OTHER REQUIREMENTS

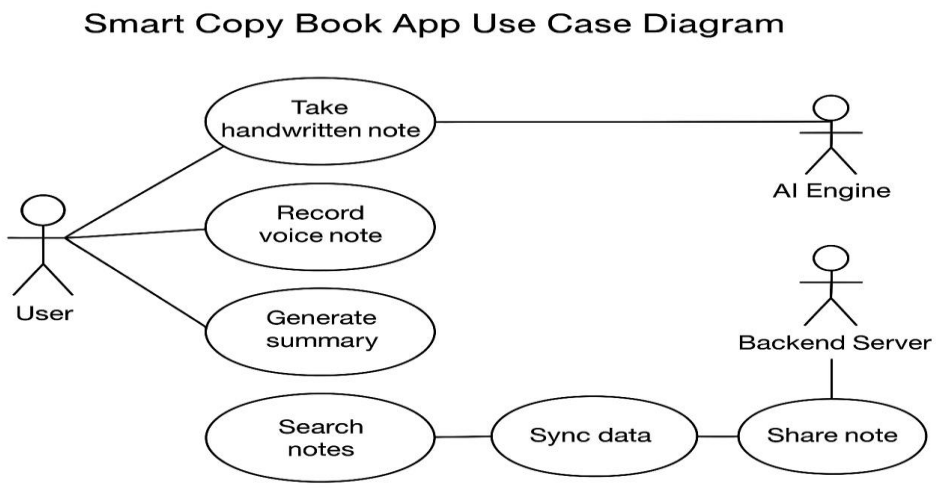
- GDPR and CCPA compliance.
- Export to PDF and TXT formats.

Appendix A: Glossary

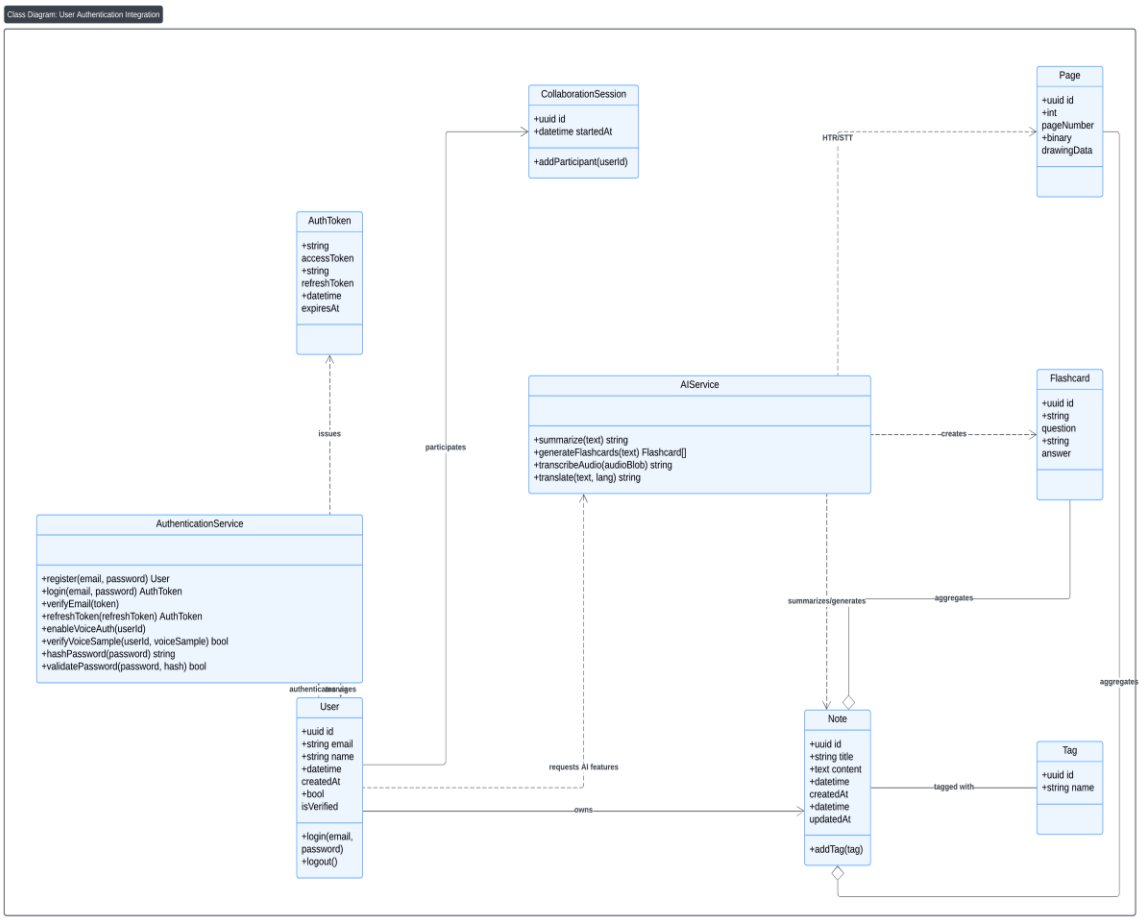
HTR – Handwriting Text Recognition
TTS – Text-to-Speech
STT – Speech-to-Text
AI – Artificial Intelligence
API – Application Programming Interface

Appendix B: Analysis Models

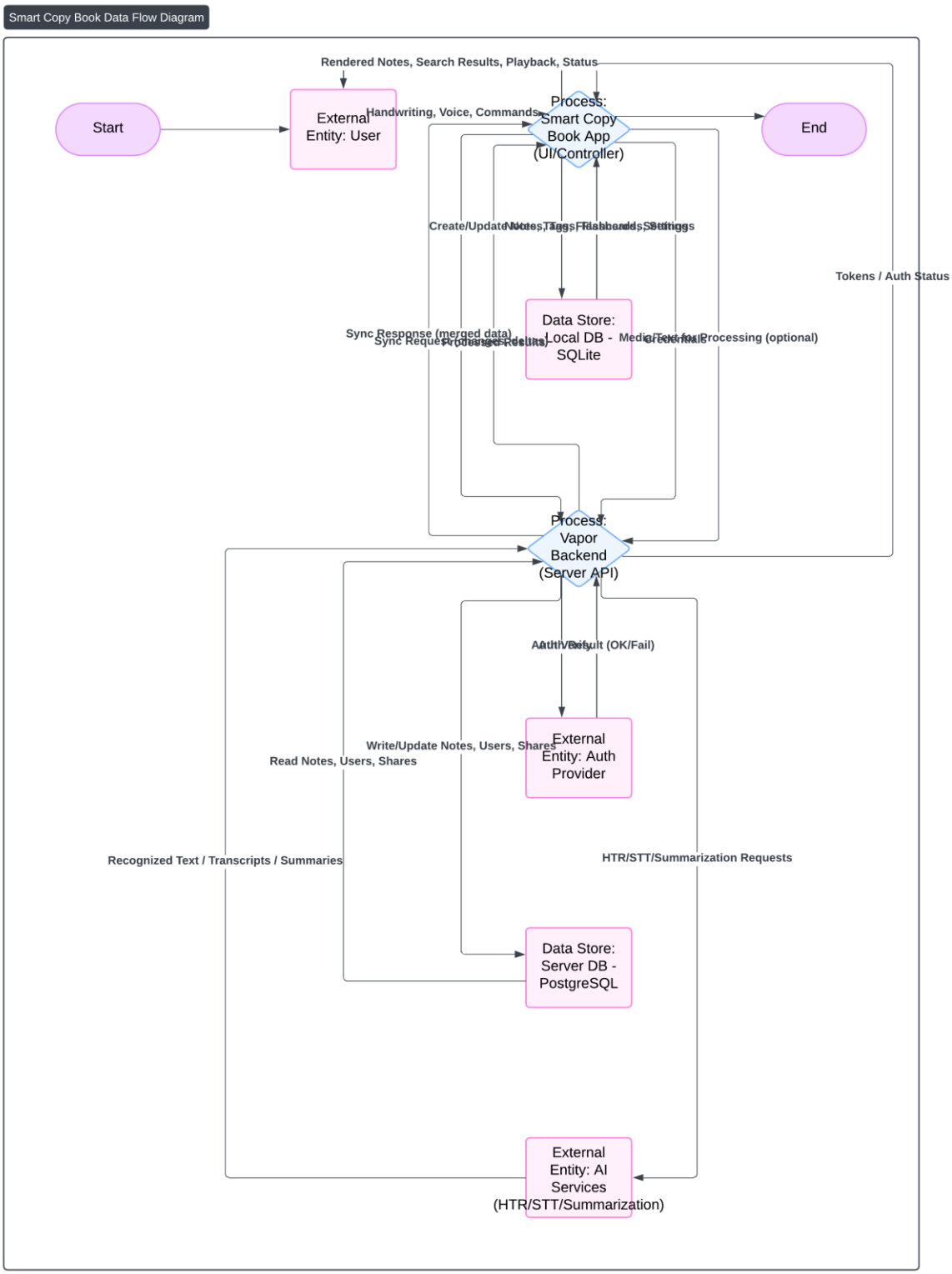
- Use Case Diagram: user interactions (handwriting, voice, summarization, sync).



- Class Diagram: entities (User, Note, Page, Flashcard, AIService).

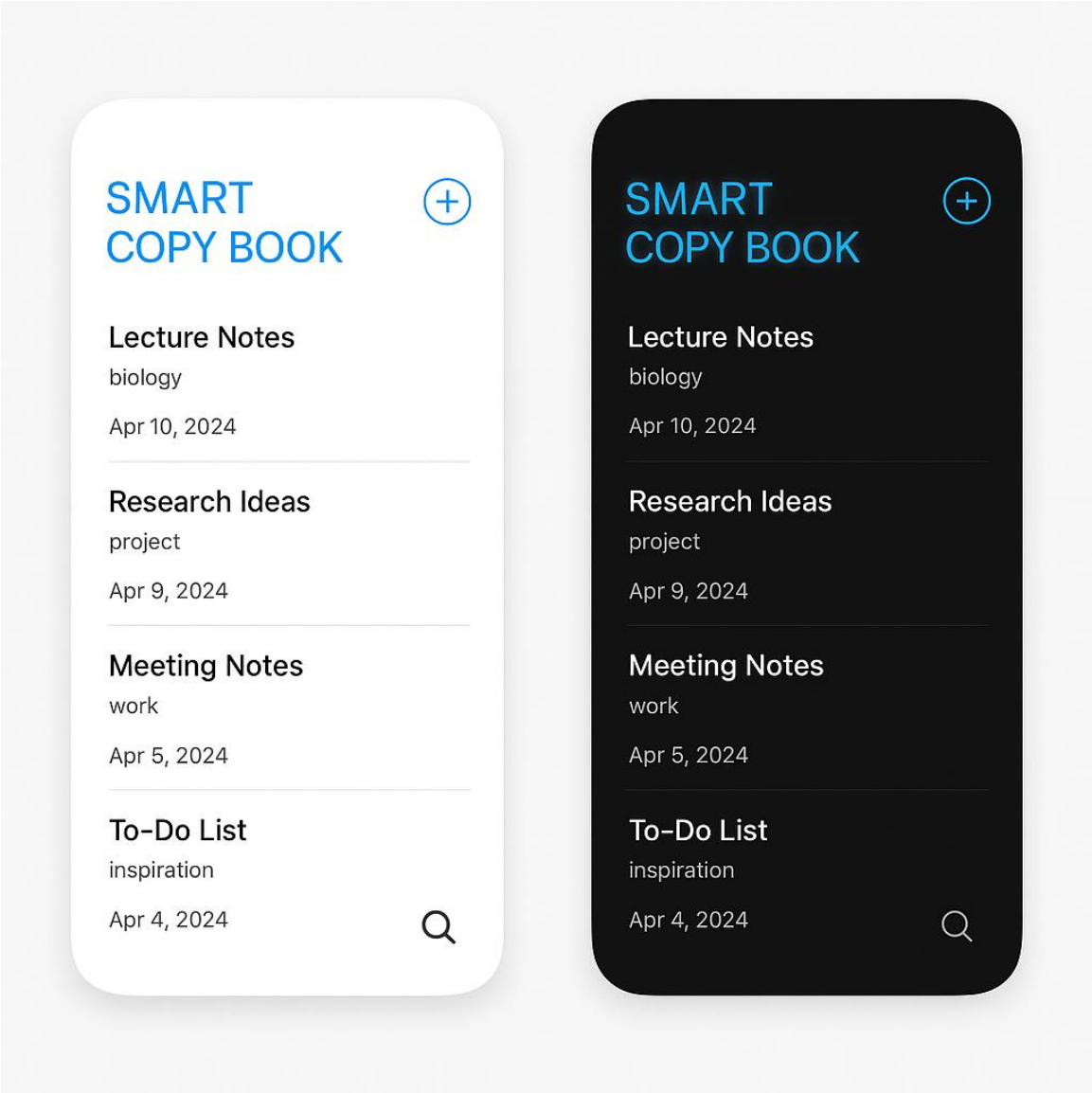


- Data Flow Diagram: data movement between UI, local DB, and server.



Appendix C: Issues List

Offline accuracy – On-device handwriting recognition may vary.

Voice authentication API – Integration with external provider to be finalized.

Collaboration performance – WebSocket latency testing required.